# LAPORAN PRAKTIKUM
# PEMROGRAMAN BERORIENTASI OBJEK
# MODUL [6] – [Inheritance]




NAMA : Abdullah Ahmad Izzah

NIM : 103132430024

KELAS : S1SD-05-01

**PROGRAM STUDI SAINS DATA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

A. Dasar Teori

Inheritance atau pewarisan merupakan salah satu konsep utama dalam Pemrograman Berorientasi Objek yang memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain. Kelas yang memberikan pewarisan disebut superclass atau kelas induk, sedangkan kelas yang menerima pewarisan disebut subclass atau kelas turunan. Dalam Java, pewarisan diterapkan menggunakan keyword extends. Tujuan inheritance adalah mengurangi duplikasi kode dan memungkinkan kelas yang lebih spesifik menggunakan karakteristik yang telah dimiliki kelas induknya.

Superclass dan subclass memiliki hubungan yang menunjukkan bahwa subclass merupakan bentuk yang lebih khusus dari superclass. Subclass dapat menggunakan anggota kelas induk yang dapat diakses sesuai aturan akses Java serta menambahkan atribut dan method baru sesuai kebutuhan. Sebagai contoh, kelas model umum dapat menjadi superclass bagi kelas model regresi dan model klasifikasi yang memiliki karakteristik tambahan masing-masing.

1. [Inheritance]

File gd1 A.java

```
package guided.gd1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

File gd1 B.java

```
package guided.gd1;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z))= " + hasil);
    }

}
```

File gd1 Demo.java

```
package guided.gd1;

public class Demo {
    public static void main(String[] args){
        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Project_Modul
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z))= 8
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
```

2. [Keyword Super]

File gd2 Employee.java

```
package guided.gd2;

public class Employee {
    private String nama;
    String department;

    public Employee(String nama){
        this.nama = nama;
    }

    public void tampilNama(){
        System.out.println("nama: " + nama);
```

```
    }
}
```

File gd2 Manager.java

```
package guided.gd2;

public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("Department: " + super.department);

    }

}
```

File gd2 Demo.java

```
package guided.gd2;

public class Demo {
    public static void main(String[] args){
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.department = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Project_Mod
nama: Adi
alamat: Sukabirus
Department: SDM
------------------------------------------------
BUILD SUCCESS
```

3. [Overloading Method]

File gd3 Lagu.java

```
package guided.gd3;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

File gd3 Main.java

```
package guided.gd3;

public class Main {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();
        d.IsiParam("Lagu 1");
        e.IsiParam("Dunia Yang Nanti", "Raim Laode");
        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ Project_Modul_6 ---
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : Dunia Yang Nanti, pencipta : Raim Laode
------------------------------------------------------------
BUILD SUCCESS
```

4. [Overiding Method]

File gd4 A.java

```
package guided.gd4;

public class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

File gd4 B.java

```
package guided.gd4;

public class B extends A {
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

File gd4 Main.java

```
package guided.gd4;

public class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output :

```
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------
BUILD SUCCESS
------------------------------------------------
```

B. Unguided

1. Soal Unguided 1

   Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Jawaban :

File ModelKomputasi.java

```
package unguided;

public class ModelKomputasi {
    private String namaModel;
    private int jumlahDataLatih;

    public ModelKomputasi(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel
                + " | Data latih: " + jumlahDataLatih);
    }

    // Menghitung biaya dasar
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Method overloading: biaya berdasarkan epoch
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

File ModelRegresi.java

```
package unguided;

public class ModelRegresi extends ModelKomputasi{

    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
```

```
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Method overloading dengan dua parameter
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

File ModelKlasifikasi.java

```
package unguided;

public class ModelKlasifikasi extends ModelKomputasi {

    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih,
            int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

File ModelDeepLearning.java

```
package unguided;
```

```
public class ModelDeepLearning extends ModelKlasifikasi {

    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih,
                             int jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

File Main.java

```
package unguided;

public class Main {

    public static void main(String[] args) {

        // Membuat array dengan tipe kelas induk
        ModelKomputasi[] daftarModel = {
            new ModelRegresi("Regresi Linier", 500, 4),
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;

        // Menampilkan informasi dan biaya setiap model
        for (ModelKomputasi model : daftarModel) {
            model.tampilkanInfo();

            int biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");

            totalBiaya += biaya;
        }
```

```
        // Total biaya seluruh model
        System.out.println("Total biaya: " + totalBiaya);

        // Biaya 10 epoch untuk Neural Network
        System.out.println("Biaya 10 epoch (Neural Network): "
                + daftarModel[2].hitungBiaya(10));

        // Biaya 10 epoch untuk Regresi Linier
        System.out.println("Biaya 10 epoch (Regresi Linier): "
                + daftarModel[0].hitungBiaya(10));

        // Biaya 10 epoch dengan batch 50 untuk Regresi Linier
        ModelRegresi regresi = (ModelRegresi) daftarModel[0];

        System.out.println(
                "Biaya 10 epoch batch 50 (Regresi Linier): "
                + regresi.hitungBiaya(10, 50)
        );
    }
}
```

Output :

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------
BUILD SUCCESS
```

Penjelasan :

Program ini terdiri dari lima class, yaitu ModelKomputasi, ModelRegresi, ModelKlasifikasi, ModelDeepLearning, dan Main. Class ModelKomputasi berfungsi sebagai superclass yang menyimpan nama model dan jumlah data latih. Class ini memiliki method tampilkanInfo() untuk menampilkan informasi model dan dua versi

method hitungBiaya(), yaitu tanpa parameter dengan rumus jumlah data latih dikalikan dua, serta dengan parameter epoch dengan rumus jumlah data latih dikalikan epoch.

Class ModelRegresi mewarisi atribut dan method dari ModelKomputasi menggunakan keyword extends. Constructor menggunakan super() untuk memanggil constructor kelas induk, sedangkan atribut jumlahFitur menyimpan jumlah fitur model regresi. Method tampilkanInfo() dan hitungBiaya() di-override agar dapat menampilkan informasi tambahan dan menghitung biaya dasar ditambah sepuluh kali jumlah fitur. Class ini juga memiliki method hitungBiaya(int epoch, int ukuranBatch) untuk menghitung biaya berdasarkan jumlah data latih, epoch, dan ukuran batch.

Class ModelKlasifikasi juga mewarisi ModelKomputasi dengan menambahkan atribut jumlahKelas. Method tampilkanInfo() di-override untuk menampilkan jumlah kelas, sedangkan hitungBiaya() di-override agar hasil biaya dasar dikalikan jumlah kelas. Sementara itu, class ModelDeepLearning mewarisi ModelKlasifikasi dan menambahkan jumlah layer. Class ini menggunakan super() untuk memanggil constructor kelas induknya serta melakukan overriding pada tampilkanInfo() dan hitungBiaya(). Biaya model deep learning diperoleh dari biaya versi klasifikasi ditambah dua ratus kali jumlah layer.

Pada class Main, ketiga objek dibuat dan dimasukkan ke dalam array bertipe ModelKomputasi. Penggunaan array ini memungkinkan seluruh objek diproses melalui perulangan yang sama. Ketika tampilkanInfo() dan hitungBiaya() dipanggil, Java menjalankan implementasi yang sesuai dengan kelas objek sebenarnya. Konsep ini merupakan penerapan overriding dan polymorphism. Program kemudian menjumlahkan biaya masing-masing model sehingga memperoleh total biaya sebesar 16.440. Pada bagian akhir, program menghitung biaya sepuluh epoch untuk Neural Network dan Regresi Linier serta biaya sepuluh epoch dengan ukuran batch 50 untuk Regresi Linier. Method dengan satu parameter tetap menggunakan implementasi dari kelas induk, sedangkan method dengan dua parameter merupakan overload tambahan pada kelas regresi.

C. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa inheritance memungkinkan suatu kelas mewarisi atribut dan method dari kelas induknya sehingga struktur program menjadi lebih terorganisasi dan mengurangi duplikasi kode. Keyword super digunakan untuk memanggil constructor dan method superclass, sedangkan method overloading memungkinkan penggunaan nama method yang sama dengan parameter berbeda. Method overriding memungkinkan setiap subclass memiliki implementasi perhitungan biaya yang sesuai dengan karakteristik modelnya. Penggunaan array bertipe

superclass juga menunjukkan penerapan polymorphism karena setiap objek dapat diproses melalui method yang sama, tetapi menghasilkan perilaku berbeda. Program berhasil menghitung biaya komputasi Regresi Linier sebesar 1.040, Decision Tree sebesar 4.800, dan Neural Network sebesar 10.600, dengan total biaya sebesar 16.440.